Lieferroboter in CARLA starten

Schritt-für-Schritt-Anleitung für Starship und Clevon: vom Repository über CARLA bis ROS 2

Stand 23.10.2026 · CARLA 0.10.0 · Unreal Engine 5.5.4 · ROS 2 Jazzy · Ubuntu 24.04 · Repository: git.ieem-ka.de/rari1014/last_mile-delivery-carla, Branch `clevon-starship-carla-0.10`

So liest du die Anleitung. Alles in den dunklen Kästen kannst du kopieren und im Terminal einfügen. Graue Zeilen mit `#` sind Kommentare. Die farbigen Schilder wie **TERMINAL 1** sagen dir, in welchem Terminal-Fenster ein Befehl läuft. Programme, die weiterlaufen sollen (CARLA, Fahrclient, Sensoren), brauchen jeweils **ein eigenes Terminal**.

1 Terminal-Grundlagen
2 Repository herunterladen
3 Einmalige Vorbereitung
4 Starship in CARLA importieren
5 CARLA starten und beenden
6 Starship: fahren und Sensoren
7 Starship über ROS 2
8 Clevon: fahren und Sensoren
9 Clevon über ROS 2
10 ROS-2-Demo: Slalom fahren
11 Probleme und Lösungen
12 Spickzettel

Das Prinzip in einem Bild

1 · CARLA
Der Simulator mit der Stadt Town10HD. Läuft die ganze Zeit.

2 · Fahrzeug
Ein Python-Programm setzt den Bot in die Stadt und lässt dich fahren.

3 · Sensoren
Weitere Programme bringen Kameras, Radar usw. an und zeigen die Daten.

4 · ROS 2
Daten und Steuerung für andere Programme bereitstellen.

1 Terminal-Grundlagen

Das Terminal ist ein Fenster, in das du Befehle tippst. Öffnen: `Strg`+`Alt`+`T`. Ein weiteres Terminal als Reiter im selben Fenster: `Strg`+`Umschalt`+`T`.

Befehl / Taste	Bedeutung
`pwd`	Zeigt, in welchem Ordner du gerade bist.
`ls`	Listet die Dateien und Ordner im aktuellen Ordner.
`mkdir ordner`	Legt einen neuen Ordner an.
`cd ordner`	Wechselt in einen Ordner. `cd ..` eine Ebene zurück, `cd ~` in deinen Home-Ordner.
`~`	Kurzform für deinen Home-Ordner, hier `/home/ieem`.
`Tab`	Vervollständigt Datei- und Ordnernamen. Spart Tippfehler.
`Pfeil hoch`	Holt den letzten Befehl zurück.
`Strg`+`Umschalt`+`V`	Einfügen im Terminal (`Strg`+`V` allein geht dort nicht).
`Strg`+`C`	Beendet das Programm, das gerade im Terminal läuft.

Lange Befehle mit `\` am Zeilenende gehören zusammen. Kopiere alle Zeilen auf einmal und füge sie ein.

2 Repository herunterladen

Das Repository enthält beide Bots: `01_Clevon1` und `02_starship_emodell`. Es ist öffentlich; eine Anmeldung ist nicht nötig.

2.1 Einmalig: Git und Git LFS

```
# Prüfen, ob Git und Git LFS da sind (Versionsnummer = installiert)
git --version
git lfs version

# Falls etwas fehlt (fragt nach deinem Passwort):
sudo apt install git git-lfs
git lfs install
```

2.2 Ordner anlegen und Repository klonen

```
# Ordner anlegen und hineinwechseln
mkdir -p ~/carla_projekt
cd ~/carla_projekt

# Repository mit beiden Bots holen (Modelldateien kommen über Git LFS mit)
git clone -b clevon-starship-carla-0.10 \
  https://git.ieem-ka.de/rari1014/last_mile-delivery-carla.git

cd last_mile-delivery-carla
git lfs pull
ls
```

Du solltest `01_Clevon1`, `02_starship_emodell` und `README.md` sehen. Alle weiteren Befehle gehen davon aus, dass das Repo unter `~/carla_projekt/last_mile-delivery-carla` liegt. Wer einen anderen Ordner wählt, ersetzt diesen Pfad überall.

```
# Später: neuesten Stand holen
cd ~/carla_projekt/last_mile-delivery-carla
git pull
git lfs pull
```

3 Einmalige Vorbereitung

> **Auf dem Institutsrechner ist das meiste bereits eingerichtet:** CARLA 0.10.0 unter `~/CarlaUE5`, Unreal Engine unter `~/UnrealEngine5_carla`, Python-Umgebung unter `~/CarlaUE5/.venv`, beide Fahrzeuge registriert, ROS 2 Jazzy installiert. Dort kannst du 3.1 und Kapitel 4 überspringen.

3.1 ROS 2 Jazzy installieren (nur falls nötig)

```
# Zeigt "jazzy" an -> fertig. Sonst installieren (dauert, fragt nach Passwort):
ls /opt/ros
bash ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/install_ros_jazzy.sh
```

3.2 Python-Paket für den ROS-Ergänzungsknoten

```
~/CarlaUE5/.venv/bin/python -m pip install pyyaml
```

3.3 Steuernachricht `carla_msgs` bauen (je Fahrzeug einmal)

Damit ROS 2 Fahrbefehle an CARLA schicken kann. Ohne `sudo`, unter einer Minute.

```
bash ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/ros_ws/build.sh
bash ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik/ROS2/native/ros_ws/build.sh
```

3.4 Schnelltest: Findet Python CARLA?

```
~/CarlaUE5/.venv/bin/python -c "import carla; print('CARLA-Python ok')"
```

4 Starship in CARLA importieren (nur auf neuem Rechner)

Ist `vehicle.starship.bot` in CARLA noch nicht vorhanden, baut ein Skript den Bot im laufenden Unreal Editor automatisch auf: Modell, Kollisionskörper, sechs Räder, Fahrzeug-Blueprint und Anmeldung bei CARLA.

1. Unreal Editor mit dem CARLA-Projekt starten (siehe unten) und warten, bis er offen ist.
2. Im Editor einmalig: *Edit → Project Settings → Plugins → Python → Enable Remote Execution* anhaken.
3. Import-Skript ausführen.

TERMINAL 1

```
# Editor starten (Ladezeit einige Minuten)
~/UnrealEngine5_carla/Engine/Binaries/Linux/UnrealEditor \
  ~/CarlaUE5/Unreal/CarlaUnreal/CarlaUnreal.uproject > /dev/null 2>&1 &

# Starship importieren und registrieren (ca. 5-7 min, endet mit SUCCESS)
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/scripts/unreal_editor/ue5
./build_all.sh

# Bordstein-Assistent anhängen (nur wenn das CARLA-Plugin die Komponente enthält)
python3 ue_remote.py s08_curb_assist_component.py

# Test im Editor: Play starten (beenden mit s07_stop.py)
python3 ue_remote.py s07_play.py
```

> **Bordstein-Assistent:** Die C++-Komponente liegt unter `02_starship_emodell/unreal/ue5_carla_plugin`. Sie muss einmalig in das CARLA-Plugin kopiert und der Editor neu gebaut werden (10–20 min). Ohne sie fährt der Bot normal, kommt aber nicht auf Bordsteine. Details: `02_starship_emodell/docs/IMPLEMENTIERUNG_CARLA_0.10.md`.

5 CARLA starten und beenden

CARLA ist der Simulator. Er muss laufen, bevor du einen Bot startest. Wähle **eine** der Varianten. Ein offener Unreal Editor (Kapitel 4) mit Play geht auch, verbraucht aber mehr Grafikspeicher.

Variante A: mit Fenster, ohne ROS 2

TERMINAL 1

```
~/UnrealEngine5_carla/Engine/Binaries/Linux/UnrealEditor \
  ~/CarlaUE5/Unreal/CarlaUnreal/CarlaUnreal.uproject \
  /Game/Carla/Maps/Town10HD_Opt -game -windowed \
  -ResX=1280 -ResY=720 -quality-level=Low -prefernvidia
```

Variante B: mit Fenster und ROS 2

TERMINAL 1

```
~/UnrealEngine5_carla/Engine/Binaries/Linux/UnrealEditor \
  ~/CarlaUE5/Unreal/CarlaUnreal/CarlaUnreal.uproject \
  /Game/Carla/Maps/Town10HD_Opt -game -windowed \
  -ResX=1280 -ResY=720 -quality-level=Low -prefernvidia --ros2
```

Variante C: ohne Fenster, mit ROS 2 (spart Grafikspeicher)

TERMINAL 1

```
~/UnrealEngine5_carla/Engine/Binaries/Linux/UnrealEditor \
  ~/CarlaUE5/Unreal/CarlaUnreal/CarlaUnreal.uproject \
  /Game/Carla/Maps/Town10HD_Opt -game -RenderOffScreen \
  -quality-level=Low -prefernvidia --ros2
```

> **Warten:** Der Start dauert 1–3 Minuten. **Wichtig:** `--ros2` mit **zwei** Bindestrichen, sonst bleibt ROS 2 aus. **Beenden:** im Terminal 1 `Strg` + `C`, vorher alle anderen Programme beenden.

```
# Läuft CARLA? (in einem neuen Terminal)
~/CarlaUE5/.venv/bin/python -c "import carla; c=carla.Client('localhost',2000); \
c.set_timeout(5); print('CARLA läuft:', c.get_world().get_map().name)"
```

6 Starship: fahren und Sensoren

Voraussetzung: CARLA läuft (Kapitel 5, Variante A oder B).

6.1 Bot spawnen und selbst fahren

TERMINAL 2

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik
~/CarlaUE5/.venv/bin/python manual_control_starship.py
```

Alternativ das unveränderte CARLA-Beispiel (ohne GNSS/IMU-Erweiterung):

```
cd ~/CarlaUE5/PythonAPI/examples
~/CarlaUE5/.venv/bin/python manual_control.py --filter 'vehicle.starship.bot'
```

Taste (im Fahrfenster)	Funktion
`W` / `S`	Gas / Bremse
`A` / `D`	Lenken links / rechts
`Q`	Rückwärtsgang ein/aus
`Leertaste`	Handbremse
`P` / `Tab` / `H`	Autopilot / Kameraposition / Hilfe
`Esc`	Beenden (Bot wird entfernt)

Der Starship fährt höchstens ca. 6 km/h. Bordsteine nimmt er senkrecht mit Vollgas; bleibt er hängen: kurz zurücksetzen und erneut anfahren.

6.2 Sensoranzeigen (je in einem eigenen Terminal)

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik
~/CarlaUE5/.venv/bin/python starship_cameras.py        # 9 Kameras in Reitern
~/CarlaUE5/.venv/bin/python starship_tof.py            # 3 ToF-Tiefenkameras
~/CarlaUE5/.venv/bin/python starship_radars.py         # 4 Radare
~/CarlaUE5/.venv/bin/python starship_ultrasonic.py     # 8 Ultraschallsensoren
```

Immer nur **eine** Anzeige pro Terminal. Tasten: `1`…`9` / `Tab` Reiter, `S` Messdaten speichern, `Esc` schließen. Nicht mehrere Kamera- oder ToF-Fenster gleichzeitig öffnen (Grafikspeicher); bei Problemen `starship_cameras.py --preview`.

6.3 Kamera folgt dem Bot (nur mit CARLA-Fenster)

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik
~/CarlaUE5/.venv/bin/python starship_follow_cam.py        # --view top oder --view side
```

7 Starship über ROS 2

CARLA veröffentlicht die Sensordaten selbst über ROS 2. Voraussetzung: CARLA mit `--ros2` (Kapitel 5, Variante B oder C) und Kapitel 3.3 erledigt. Alle Topics liegen unter `/carla/starship/…`.

7.1 Bot spawnen: selbst fahren oder für ROS-Steuerung

TERMINAL 2

```
# selbst fahren:
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik
~/CarlaUE5/.venv/bin/python manual_control_starship.py

# ODER ohne Fahrclient (für 7.5); entfernen mit --remove
~/CarlaUE5/.venv/bin/python \
  ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/spawn_starship.py
```

7.2 Sensoren für ROS 2 anbringen

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2
~/CarlaUE5/.venv/bin/python starship_ros_sensors.py
# nur einzelne Gruppen: ... --groups nav radar   (nav, radar, tof, cameras)
```

7.3 Ergänzungsknoten: Ultraschall, Odometrie, Fahrzeuglage

TERMINAL 4

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2
source ros_ws/setup.bash
~/CarlaUE5/.venv/bin/python starship_ros_extras.py
```

7.4 Prüfen, ob Daten ankommen

TERMINAL 5

```
source /opt/ros/jazzy/setup.bash
ros2 topic list | grep starship                     # alle Starship-Topics
ros2 topic hz /carla/starship/starship_gnss         # Rate (ca. 5 Hz), Strg+C
ros2 topic echo --once /carla/starship/starship_gnss  # eine Nachricht
ros2 run tf2_ros tf2_echo map starship              # Position des Bots
```

7.5 Über ROS 2 steuern

Nur mit einem Bot aus `spawn_starship.py`; ein laufender Fahrclient überschreibt die ROS-Befehle.

TERMINAL 5

```
source ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/ros_ws/setup.bash

# Geradeaus fahren (sendet 10x pro Sekunde, Strg+C beendet das Senden)
ros2 topic pub -r 10 /carla/starship/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{throttle: 0.7}'

# Linkskurve (steer: -1 = links, +1 = rechts)
ros2 topic pub -r 10 /carla/starship/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{throttle: 0.6, steer: -1.0}'

# Anhalten (einmalig senden, wartet auf die Verbindung)
ros2 topic pub --once -w 1 /carla/starship/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{brake: 1.0}'
```

> **Kein Not-Halt:** Hörst du mit dem Senden auf, fährt der Bot mit dem letzten Befehl weiter. Zum Anhalten immer ausdrücklich den Bremsbefehl senden.

8 Clevon: fahren und Sensoren

Voraussetzung: CARLA läuft (Kapitel 5, Variante A oder B).

8.1 Clevon spawnen und selbst fahren

TERMINAL 2

```
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik
~/CarlaUE5/.venv/bin/python manual_control_clevon.py
```

Alternativ: `manual_control.py --filter 'vehicle.clevon.1'` im CARLA-Beispielordner. Tasten wie beim Starship, zusätzlich `L` Licht, `Umschalt`+`L` Fernlicht, `Z`/`X` Blinker. Der Clevon fährt bis ca. 50 km/h.

8.2 Sensoranzeigen (je in einem eigenen Terminal)

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik
~/CarlaUE5/.venv/bin/python clevon_cameras.py          # 6 Kameras, je 2 pro Reiter
~/CarlaUE5/.venv/bin/python clevon_radars.py           # 2 Front-Radare
~/CarlaUE5/.venv/bin/python clevon_lidar.py            # Dach-LiDAR
```

Kameras schonender: `clevon_cameras.py --low-load` oder `--preview`. Nachführkamera: `clevon_follow_cam.py`.

9 Clevon über ROS 2

Gleiches Vorgehen wie beim Starship. Voraussetzung: CARLA mit `--ros2`. Topics unter `/carla/clevon/…`.

TERMINAL 2

```
# selbst fahren ODER ohne Fahrclient spawnen (für ROS-Steuerung):
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik
~/CarlaUE5/.venv/bin/python manual_control_clevon.py
~/CarlaUE5/.venv/bin/python ROS2/native/spawn_clevon.py
```

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik/ROS2/native
~/CarlaUE5/.venv/bin/python clevon_ros_sensors.py --preview
# Gruppen: nav radar lidar cameras;  Kameras: --preview / --low-load / ohne (Full HD)
```

TERMINAL 4

```
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik/ROS2/native
source ros_ws/setup.bash
~/CarlaUE5/.venv/bin/python clevon_ros_extras.py
```

TERMINAL 5

```
source ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik/ROS2/native/ros_ws/setup.bash
ros2 topic list | grep clevon
ros2 topic pub -r 10 /carla/clevon/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{throttle: 0.5}'
ros2 topic pub --once -w 1 /carla/clevon/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{brake: 1.0}'
```

> **Native ROS 2 oder Python-API-Bridge?** Für CARLA 0.10 werden nur die Skripte im Unterordner `01_Clevon1/Sensorik/ROS2/native/` verwendet: CARLA sendet die Sensordaten selbst. Die übrigen Dateien direkt in `01_Clevon1/Sensorik/ROS2/` (`clevon_ros_bridge.py` usw.) sind die frühere Bridge über die CARLA-Python-API für CARLA 0.9.16 (Windows/WSL, ROS 2 Humble, mit Licht- und Klappenbefehlen). Sie wird nicht mehr verwendet und darf nicht gleichzeitig laufen. Übersicht: `01_Clevon1/Sensorik/ROS2/README.md`.

10 ROS-2-Demo: Slalom fahren (beide Bots)

Für Vorführungen: Ein Skript setzt den Bot an eine lange gerade Straße, lenkt ihn über ROS 2 im Slalom (Topic `vehicle_control_cmd`) und zeigt zwei **Fahrzeugkameras aus den JSON-Sensorprofilen**, deren Bilder es selbst über ROS 2 empfängt: beim Starship Frontkamera K1

und Heckkamera K8, beim Clevon Front- und Heckkamera. Position, Sichtfeld und Rate entsprechen dem Profil. Über jedem Bild steht das ROS-Topic. Am Ende der Geraden setzt das Skript den Bot an den Start zurück; die Kamera im CARLA-Fenster folgt ihm.

Voraussetzung: CARLA mit `--ros2` und Fenster (Kapitel 5, Variante B) und Kapitel 3.3. Kein anderer Fahrclient für diesen Bot darf laufen.

> **Auch für den Clevon im Starship-Ordner bleiben!** Das Demo-Skript und sein `ros_ws` liegen nur unter `02_starship_emodell/Sensorik/ROS2`. Das Skript steuert beide Fahrzeuge; den Clevon wählst du mit `--vehicle clevon`. Im Ordner `01_Clevon1/Sensorik/ROS2` gibt es weder `ros2_slalom_demo.py` noch `ros_ws/build.sh` (Fehler *No such file or directory*).

TERMINAL 2

```
# immer im Starship-Ordner, auch für den Clevon
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2
bash ros_ws/build.sh          # nur beim ersten Mal nach dem Klonen
source ros_ws/setup.bash

# Starship (ca. 5 km/h):
~/CarlaUE5/.venv/bin/python ros2_slalom_demo.py

# ODER Clevon (ca. 15 km/h):
~/CarlaUE5/.venv/bin/python ros2_slalom_demo.py --vehicle clevon
```

Option	Wirkung
`--cameras ROLLE ROLLE`	zwei andere Kameras aus dem Profil, z. B. `clevon_camera_left_upper` `clevon_camera_right_upper`
`--duration 60`	nach 60 s automatisch beenden
`--spawn 12`	festen Spawnpunkt statt automatischer Suche
`--no-view`	ohne Kamerafenster

Beenden: `Esc` im Demo-Fenster oder `Strg`+`C`. Das Skript sendet dann einen Bremsbefehl und entfernt Bot und Kameras. Zusätzliche Sensoren und Topics wie in Kapitel 7 lassen sich parallel in weiteren Terminals starten:

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2
~/CarlaUE5/.venv/bin/python starship_ros_sensors.py --groups nav radar tof
```

TERMINAL 4

```
source ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/ros_ws/setup.bash
ros2 topic list | grep starship
ros2 topic hz /carla/starship/starship_radar_front/point_cloud
ros2 topic echo /carla/starship/vehicle_control_cmd
```

11 Probleme und Lösungen

Meldung / Verhalten	Lösung
`No such file or directory`	Falscher Ordner oder Tippfehler. Mit `pwd` und `ls` prüfen, die `cd` -Zeile erneut einfügen.
`ros2_slalom_demo.py` oder `ros_ws/build.sh` nicht gefunden (Clevon)	Die Demo liegt nur im Starship-Ordner: `cd …/02_starship_emodell/Sensorik/ROS2` , dann `--vehicle clevon` .
`time-out ... while waiting for the simulator`	CARLA läuft noch nicht oder lädt noch. Warten, Test aus Kapitel 5.
`get_trafficmanager ... std::exception`	Ein alter Fahrclient läuft noch: alle Terminals mit `Strg`+`C` beenden oder `pkill -f manual_control` .
`Out of memory on Vulkan` , CARLA stürzt ab	Grafikspeicher voll. Weniger Kameras, `--preview` , Variante C. Kein zweites CARLA und keinen Editor parallel.
Keine ROS-Topics	CARLA ohne `--ros2` gestartet oder `source …/setup.bash` vergessen. Im CARLA-Log muss „ROS2: Enabling ROS2“ stehen.
`Package 'carla_msgs' not found`	Kapitel 3.3 ausführen und vor dem Befehl `source …/ros_ws/setup.bash` .
Bot reagiert nicht auf ROS-Befehle	Läuft noch ein Fahrclient? Bot mit `spawn_starship.py` bzw. `spawn_clevon.py` neu setzen.
`No module named 'carla'` oder `'yaml'`	Immer `~/CarlaUE5/.venv/bin/python` verwenden; für `yaml` Kapitel 3.2.
`NO_EDITOR_FOUND` beim Import	Editor noch nicht fertig geladen oder Remote Execution nicht aktiviert (Kapitel 4).
`vehicle.starship.bot` nicht gefunden	Starship noch nicht importiert: Kapitel 4.
Alles hängt	Alle Programme beenden, notfalls `pkill -f UnrealEditor` und CARLA neu starten.

Reihenfolge beim Beenden

1. Sensoranzeigen und ROS-Programme: `Esc` bzw. `Strg`+`C` (Sensoren werden entfernt).
2. Fahrclient: `Esc` .
3. CARLA: im Terminal 1 `Strg`+`C` .

12 Spickzettel

Starship

Fahrzeug-ID	`vehicle.starship.bot`
Ordner	`02_starship_emodell/Sensorik`
Fahren	`manual_control_starship.py`
Anzeigen	`starship_cameras/_tof/_radars/_ultrasonic.py`
ROS	`ROS2/spawn_starship.py` , `starship_ros_sensors.py` , `starship_ros_extras.py`
Demo	`ROS2/ros2_slalom_demo.py`
Topics	`/carla/starship/…`
Sensoren	9 Kameras, 3 ToF, 4 Radar, 8 Ultraschall, GNSS, IMU

Clevon

Fahrzeug-ID	`vehicle.clevon.1`
Ordner	`01_Clevon1/Sensorik`
Fahren	`manual_control_clevon.py`
Anzeigen	`clevon_cameras/_radars/_lidar.py`
ROS	`ROS2/native/spawn_clevon.py` , `clevon_ros_sensors.py` , `clevon_ros_extras.py`
Demo	im Starship-Ordner: `02_starship_emodell/Sensorik/ROS2/ros2_slalom_demo.py --vehicle clevon`
Topics	`/carla/clevon/…`
Sensoren	6 Kameras, 2 Radar, LiDAR, GNSS, IMU

Typische Sitzung (ROS-2-Demo)

Terminal	Was läuft
TERMINAL 1	CARLA mit `--ros2` und Fenster (Kapitel 5 B)
TERMINAL 2	`ros2_slalom_demo.py` (Kapitel 10)
TERMINAL 3	Sensoren für ROS (`*_ros_sensors.py`)
TERMINAL 4	Prüfen mit `ros2 topic …`

Weiterführend im Repo: `README.md` , `02_starship_emodell/Sensorik/ROS2/README.md` , `01_Clevon1/Sensorik/ROS2/native/README.md` , `02_starship_emodell/docs/IMPLEMENTIERUNG_CARLA_0.10.md` .